

FILTERSYNTHESIZER V1.1.0

Quick Start

One filter, from a specification to a bill of materials and a PDF report, in about ten minutes.

2026-10-04

Screenshots captured at v1.1.0

Before you start

FilterSynthesizer turns an active-filter specification into buildable sections: component values on standard E-series parts, a schematic for each section, an LTspice simulation of the whole cascade, and a PDF report. It assumes you know your filter approximations; it takes care of the rest.

Start it one of two ways:

- **The packaged app:** double-click `FilterSynthesizer.exe`. A console window opens and, 10–20 seconds later on the first launch (2–4 seconds after that), your browser shows the app, normally at `http://localhost:8501`. Closing the console window quits the app.
- **From source:** `pip install -r requirements.txt`, then `python -m streamlit run app.py` in the app folder.

What you will build. This guide follows one filter from start to finish — a 5th-order Butterworth low-pass with a 2 kHz corner and a passband gain of 1.5 V/V — and every number and picture in it comes from that run. At the end you will have a bill of materials and a schematic for each section, an LTspice zip and a PDF report.

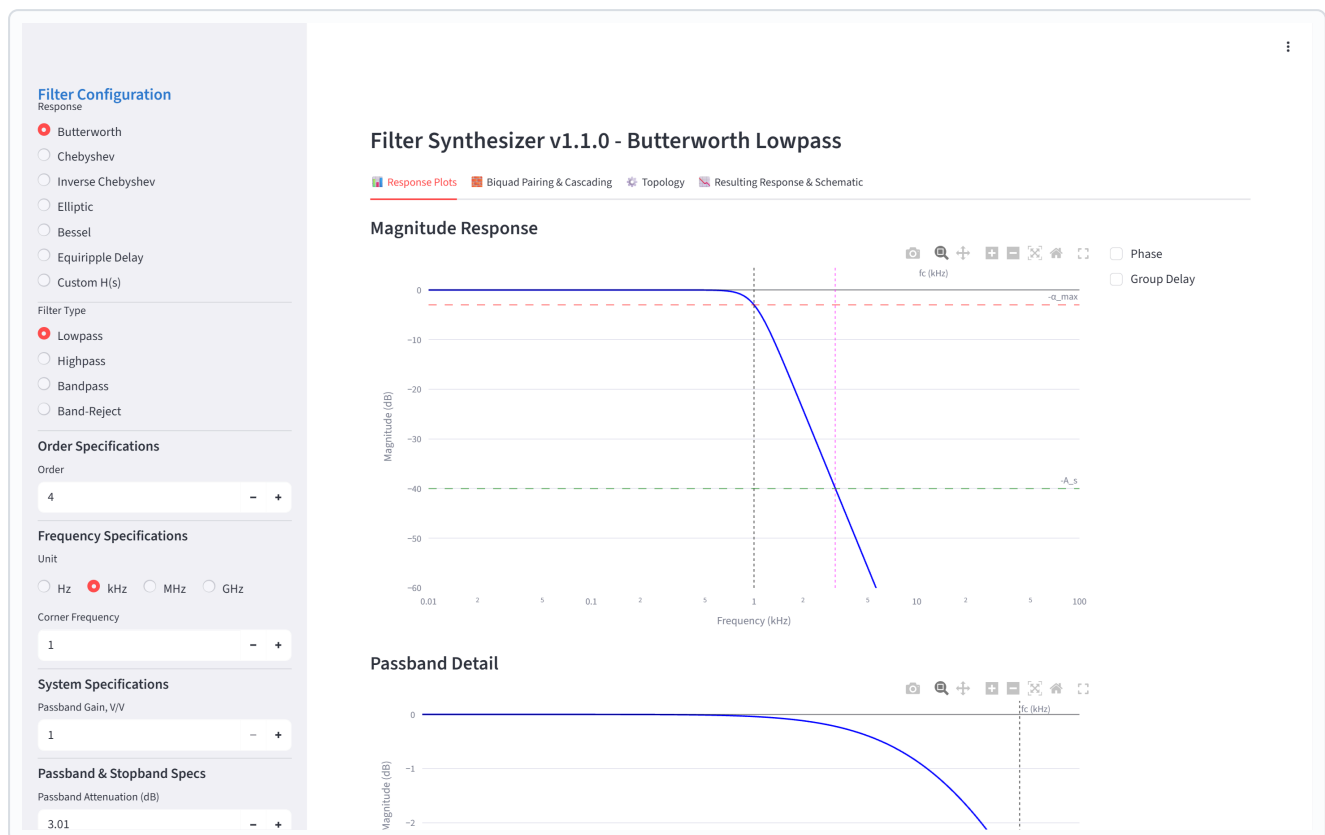


Figure 1. The app on first launch: the specification on the left, four tabs across the top.

The four tabs are a pipeline, not a menu: each works on what the previous one produced, so go left to right. Change the specification in the first two tabs, where it costs nothing — not after solving hardware.

Boxes with a blue border hold the controls that change the design; the amber box in the Topology tab is where you pick a result.

1. Specify the filter

In the sidebar, top to bottom:

- **Response** Butterworth and **Filter Type** Lowpass — both already selected.
- **Order** 5.
- **Unit** kHz — it sits above the frequency box — and **Corner Frequency** 2.
- **Passband Gain, V/V** 1.5.

Leave the other two boxes as they are:

- **Passband Attenuation** 3.01 dB — how far down the response is at the corner frequency. 3.01 dB is the classic Butterworth -3 dB corner; a different value moves the poles and changes the filter.
- **Stopband Attenuation** 40 dB — for a Butterworth this only sets where the app reports the stopband edge. The design does not depend on it.

The image shows two panels from the app's sidebar. The left panel, titled 'Filter Configuration', has three sections: 'Response' with radio buttons for Butterworth (selected), Chebyshev, Inverse Chebyshev, Elliptic, Bessel, Equiripple Delay, and Custom H(s); 'Filter Type' with radio buttons for Lowpass (selected), Highpass, Bandpass, and Band-Reject; and 'Order Specifications' with a text box for 'Order' set to 5. The right panel, titled 'Frequency Specifications', has a 'Unit' section with radio buttons for Hz, kHz (selected), MHz, and GHz; a 'Corner Frequency' section with a text box set to 2; and a 'System Specifications' section with a text box for 'Passband Gain, V/V' set to 1.5.

Figure 2. The worked example's specification, in two halves of the sidebar.

You should now see the title *Filter Synthesizer v1.1.0 - Butterworth Lowpass*, and the plots on the first tab redrawn a moment after each edit.

2. Check the approximation before you build it

The **Response Plots** tab shows the filter you specified. The passband sits at $+3.52$ dB — that is 1.5 V/V — and the curve crosses the red $-\alpha_{\text{max}}$ line exactly at the 2 kHz corner. Under the plots, the app reports the calculated stopband edge for the 40 dB target: 5.0122 kHz.

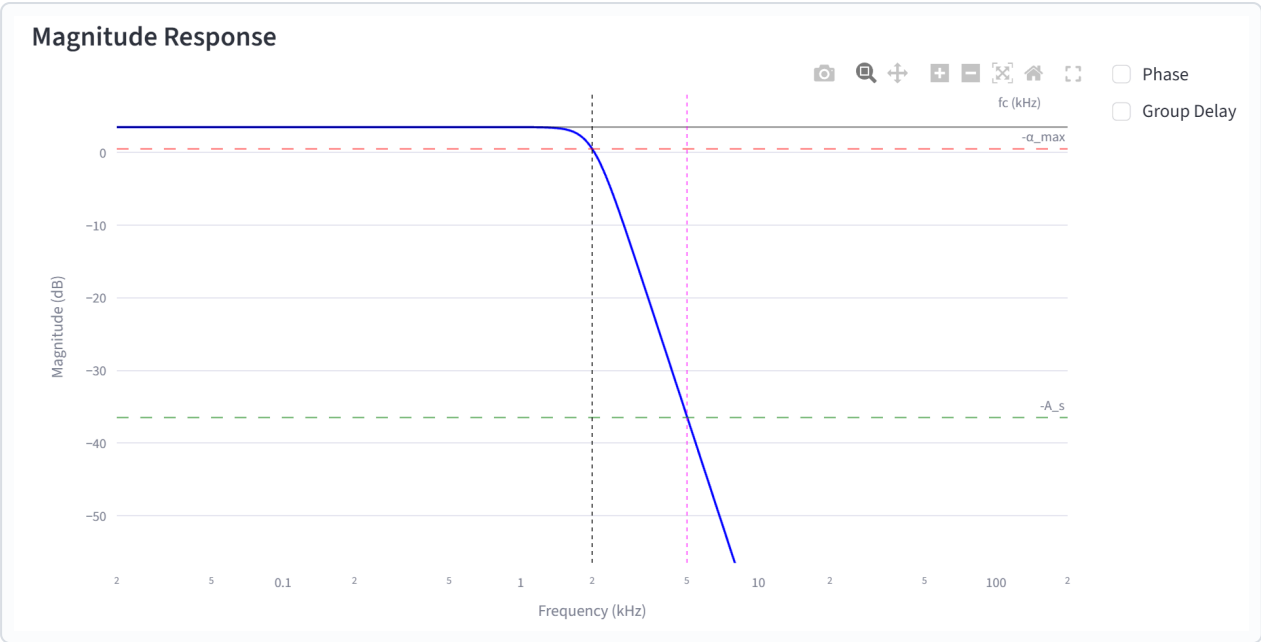


Figure 3. Response Plots: flat at +3.52 dB, and down to the red $-\alpha_{\max}$ line at the 2 kHz corner.

At the end of the same tab, the grey **Roots & Transfer Function** frame lists the poles: open **Root Locations**. Normalized to the corner, a Butterworth's poles lie on the unit circle: here one real pole at -1 and two complex pairs.

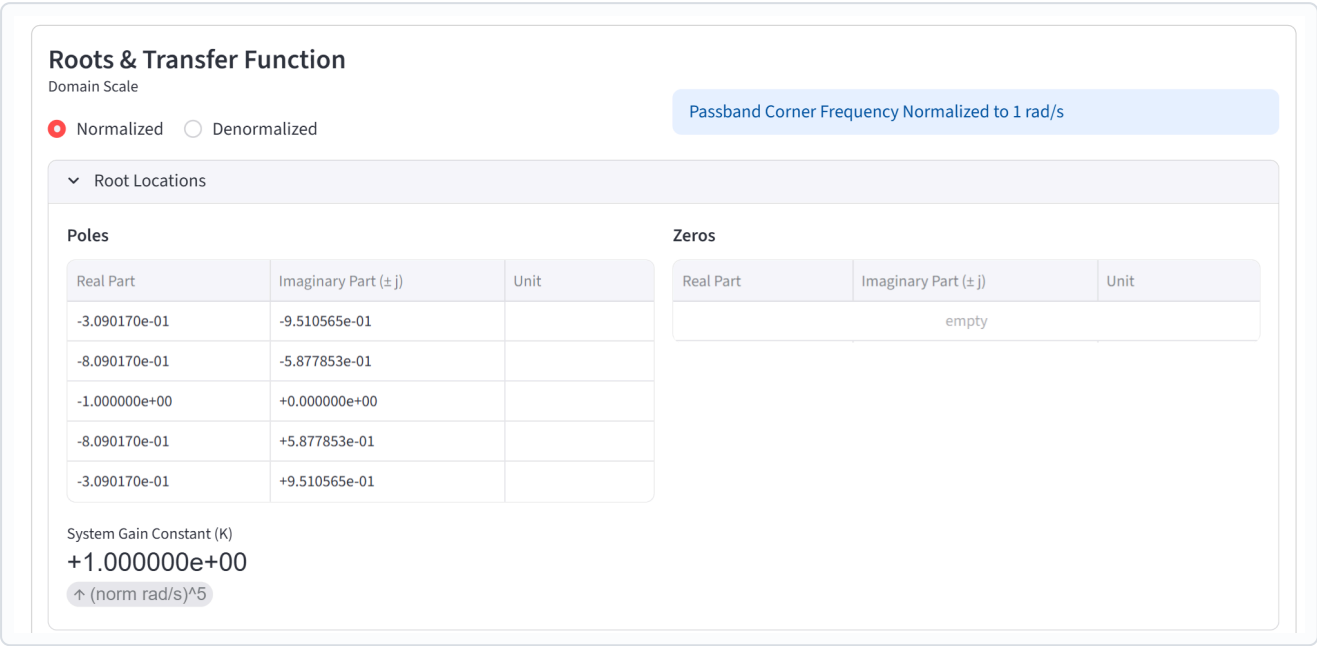


Figure 4. Root Locations: five poles, one real and two complex pairs. The real pole is what the next chapter deals with.

This is the moment to iterate. A different order or corner costs nothing now; after the next chapters, every change means solving the hardware again.

3. Split the filter into buildable sections

An active filter is built as a cascade of sections, each realizing a pair of poles — and five poles do not divide into pairs. On the **Biquad Pairing & Cascading** tab, tick **Enable 3rd-Order Sections (Absorb 1st-Order Poles)**; it is off by default. The leftover real pole then joins one of the pairs as a single 3rd-order section: two sections instead of three, and one op-amp fewer.

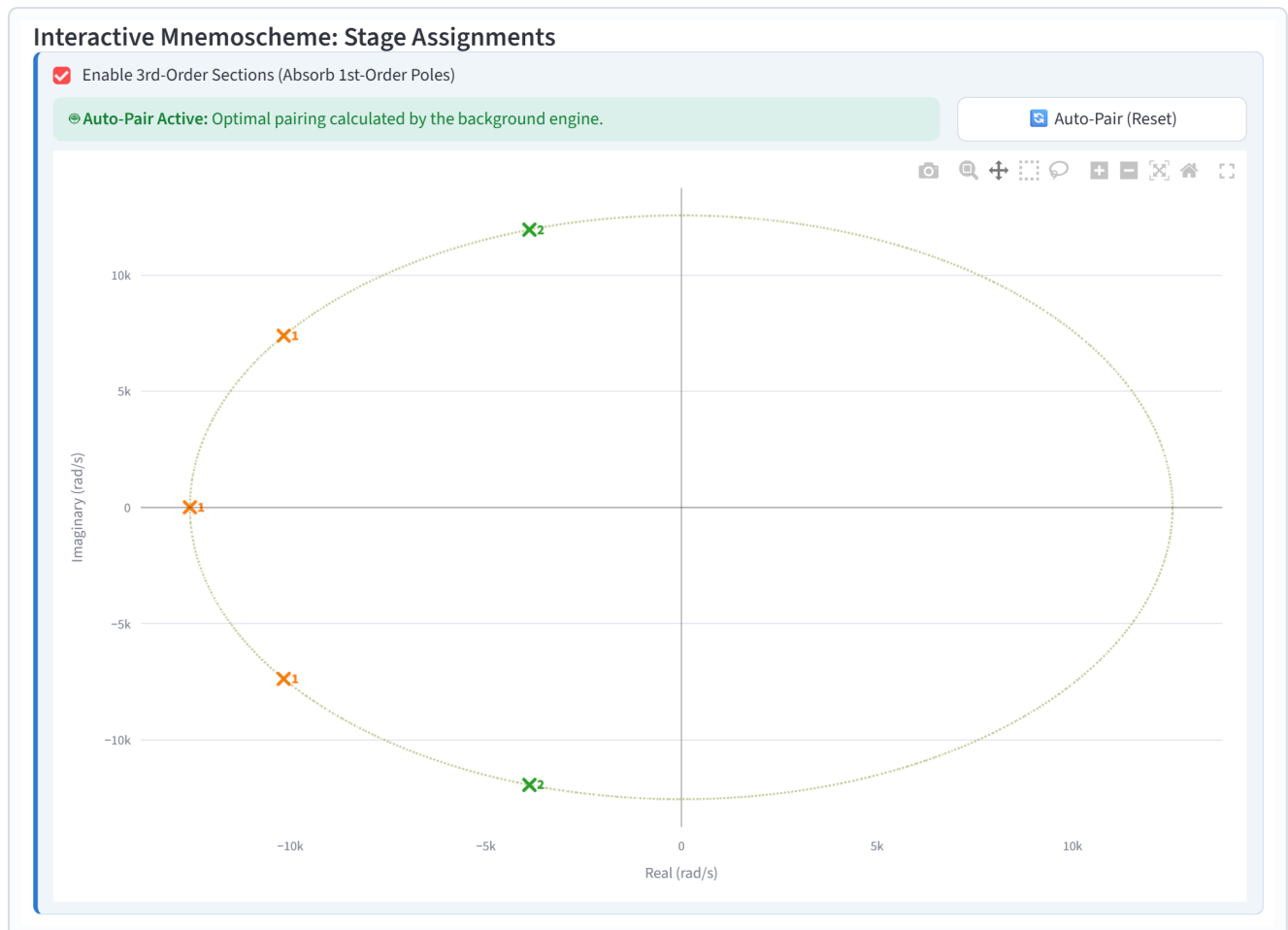


Figure 5. With absorption on, the real pole joins the low-Q pair as section 1 (orange); the high-Q pair is section 2 (green).

Sections are numbered by rising Q. Section 1 is the 3rd-order one, $Q = 0.618$ plus the real pole; section 2 is 2nd order, $Q = 1.618$. Both have $f_0 = 2$ kHz — a Butterworth's poles all sit at the corner frequency.

Next, the gain. The pairing first sets every section to unity passband gain. What is still missing to reach the specified gain is the *remaining* gain — the gain over unity, shown as **Calculated Remainder**. Here the sections give 1 V/V, so the remainder is the whole 1.5 V/V. **Remaining Gain Distribution** decides which section carries it. Choose **Apply Remaining Gain to First Stage**: section 1 gets a gain of 1.5, and section 2 stays at unity.

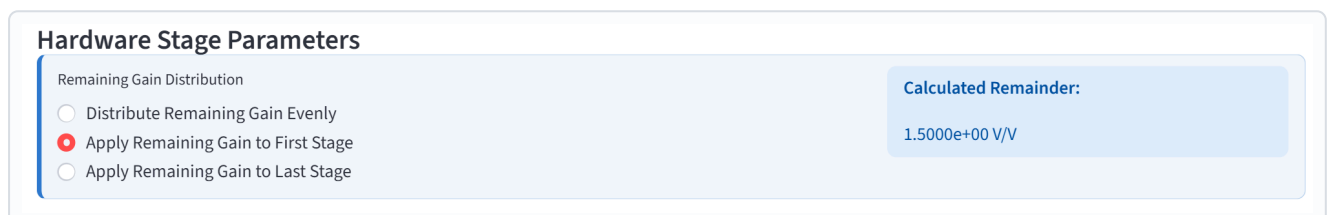


Figure 6. The whole 1.5 V/V remainder goes to the first stage.

You should now see *Calculated Remainder: 1.5000e+00 V/V*, and below it one table per section.

4. Turn section 1 into real components

The **Topology** tab lists the two sections. Each has a header line — order, real pole f_p , f_0 and Q — a settings expander, a gain control and a **Solve section** button. The caption names the circuit the app will solve:

3LP-gained for section 1, a 3rd-order low-pass with gain; **2LP-unity** for section 2.

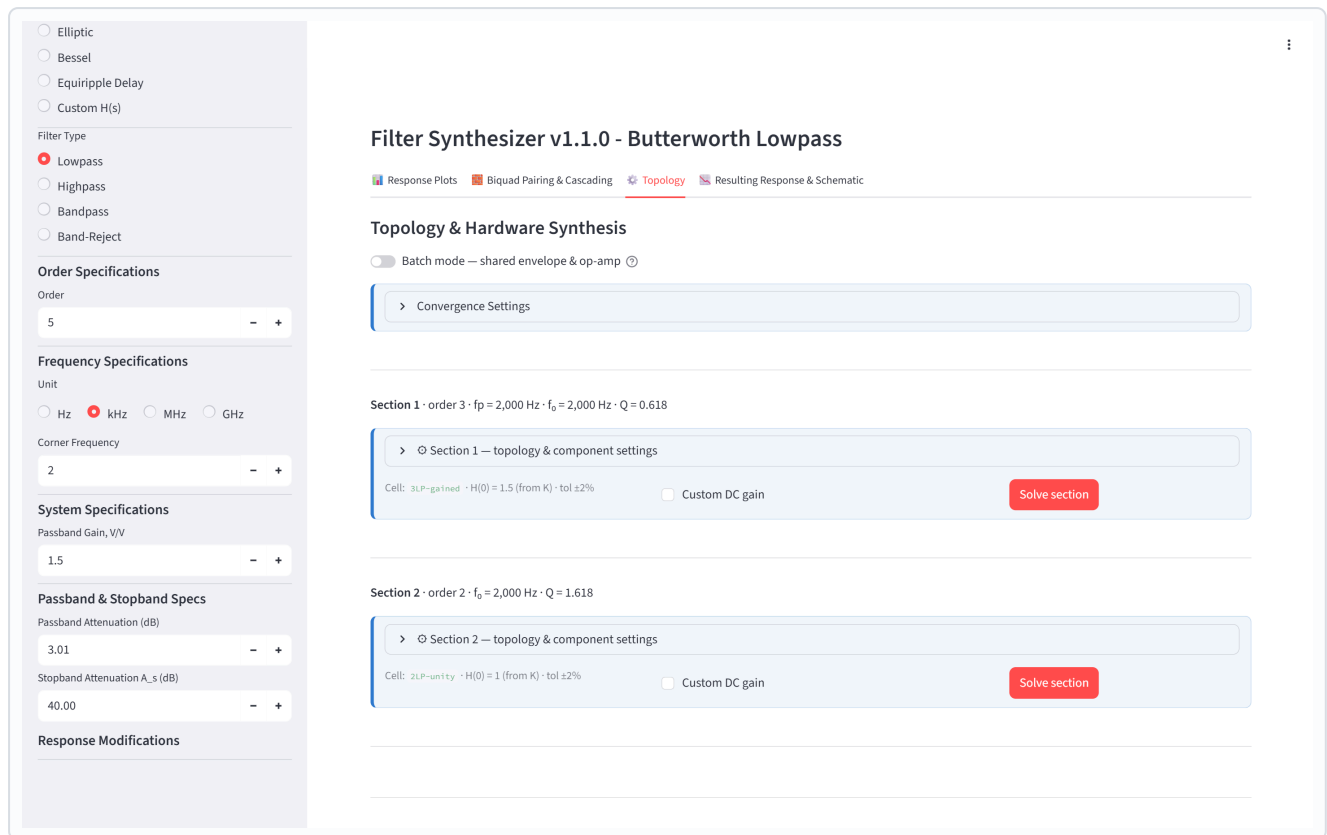


Figure 7. The Topology tab before solving. Section 1: order 3, $f_p = f_0 = 2$ kHz, $Q = 0.618$. Section 2: order 2, $Q = 1.618$.

For this run, change nothing. Leave **Batch mode** off: it shares one op-amp and one set of component limits between all sections, which you do not need here. The settings expander holds the circuit family (VCVS, i.e. Sallen-Key), the op-amp model (Ideal), the component limits (68 pF–10 nF, 300 Ω –2 M Ω) and the E-series (E12 capacitors, E48 resistors) — the User Manual, §3.4, covers each one. **Convergence Settings** at the top of the tab matters only when a solve comes back empty.

Press **Solve section** in section 1. The solve runs in the background: the button greys out, a caption says *Solving...*, and the other sections and tabs stay usable. It takes from a few seconds to about a minute, depending on the machine. Sections solve in parallel, so you could start section 2 right away.

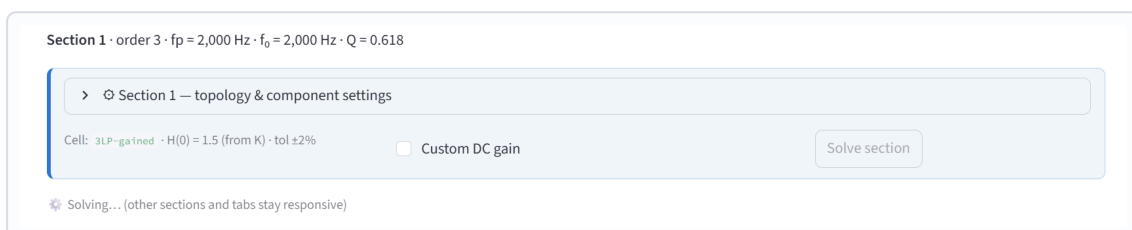


Figure 8. A solve in progress. Other sections and every other tab stay usable meanwhile.

When it finishes, a table of candidate BOMs appears, best first. They are ranked by **Sens** — how much the response moves when a component drifts; lower is better — with ties broken by **Snap cost**, how far the snapped parts leave the section from its target. **DC gain** is what the snapped parts actually give. The component columns follow the circuit: C1, C3, C4 and R1–R6 here.

Sort by

Sens score

Descending

	#	Topology	Sens	Snap cost	DC gain	C1	C3	C4	R1	R2	R3	R5	R6
☐	0	3LP-gained	2.08	0.08	1.511	10n	82p	82p	8.25k	787k	1.15M	27.4k	53.6k
☐	1	3LP-gained	2.08	0.08	1.488	10n	150p	180p	8.66k	316k	681k	27.4k	56.2k
☐	2	3LP-gained	2.08	0.09	1.488	8.2n	68p	82p	9.53k	681k	1.62M	27.4k	56.2k
☐	3	3LP-gained	2.08	0.10	1.488	10n	150p	150p	7.87k	464k	619k	27.4k	56.2k
☐	4	3LP-gained	2.08	0.11	1.488	8.2n	120p	120p	9.53k	562k	787k	27.4k	56.2k
☐	5	3LP-gained	2.08	0.11	1.488	10n	100p	100p	8.25k	681k	909k	27.4k	56.2k
☐	6	3LP-gained	2.08	0.11	1.511	8.2n	120p	150p	10k	383k	909k	27.4k	53.6k
☐	7	3LP-gained	2.08	0.11	1.511	10n	68p	82p	7.87k	715k	1.62M	27.4k	53.6k
☐	8	3LP-gained	2.08	0.12	1.487	10n	82p	100p	7.87k	590k	1.33M	26.1k	53.6k
☐	9	3LP-gained	2.08	0.13	1.487	8.2n	150p	180p	10.5k	316k	681k	26.1k	53.6k

Pick a row to select that BOM.

Figure 9. Candidate BOMs for section 1, best first.

Now the step people miss: **tick the box at the left of row 0**. Solving produces candidates; only a pick chooses one.

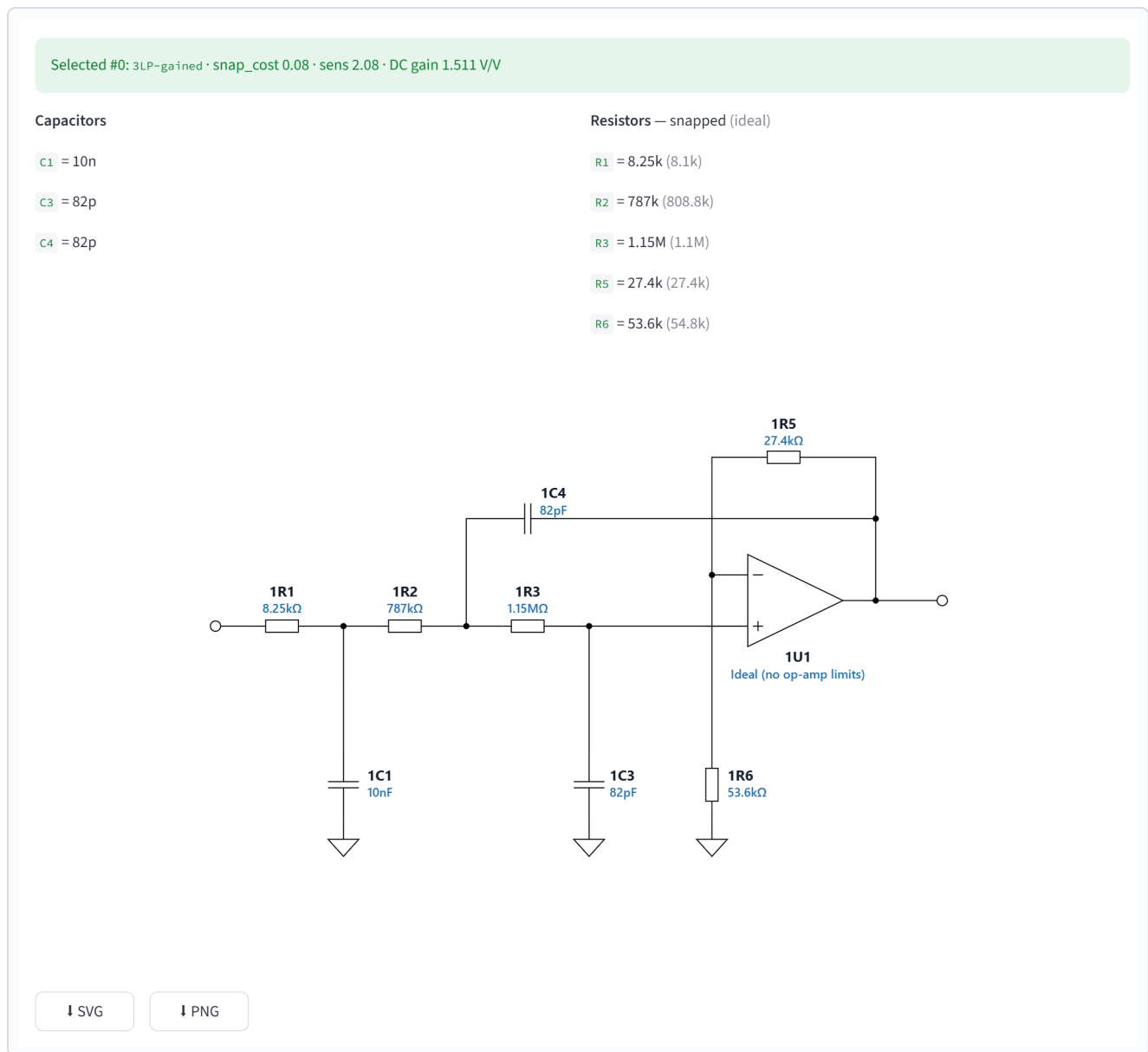
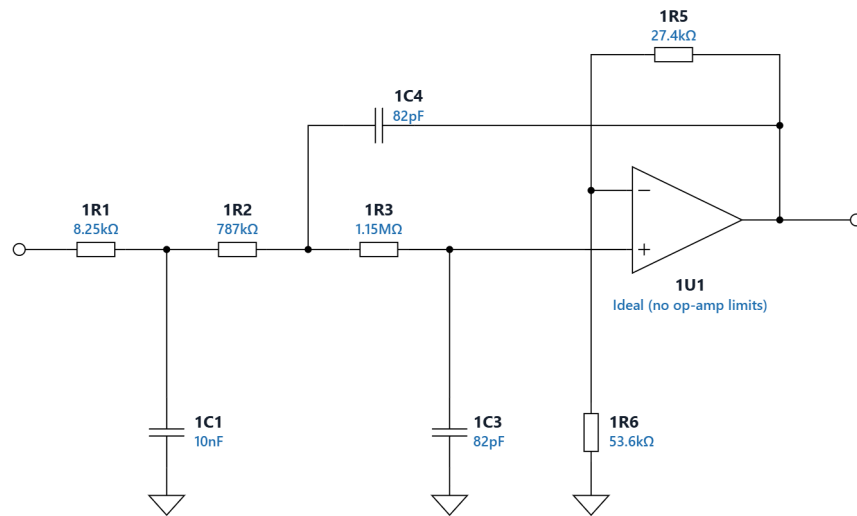


Figure 10. Row 0 picked. Capacitors come straight from E12; resistors are snapped to E48, their ideal values in grey.

The pick lists the parts and draws the schematic, every part labelled with its value. Here it is an RC pole (1R1, 1C1) in front of a Sallen-Key stage whose gain, $1 + R5/R6 = 1 + 27.4k/53.6k = 1.511$, is the 1.511 V/V in the table.



↓ SVG

↓ PNG

Figure 11. Section 1's schematic. ↓ SVG and ↓ PNG under it save the drawing.

A solved section is not a chosen section. Until every section has a picked row, the cascade view and the report stay blocked.

5. Do the same for section 2

The same three moves: **Solve section** in section 2, wait, tick **row 0**. Section 2 is a unity-gain Sallen-Key, 2LP-unity — two resistors and two capacitors.

At the foot of the tab, **Overall filter** now shows the gain of the whole cascade: about 1.51 V/V against the 1.5 specified. The 0.7 % difference is resistor snapping in section 1, whose gain came out at 1.511 instead of 1.5. A yellow box there would mean the output is inverted; two non-inverting Sallen-Key sections leave none.

6. Check what you actually built

The **Resulting Response & Schematic** tab rebuilds the whole cascade from the parts you picked. Blue is the design, red is what the snapped parts do, and wherever the two separate you see the price of E-series values. Here they overlap: the realized passband is +3.58 dB against +3.52 dB designed.

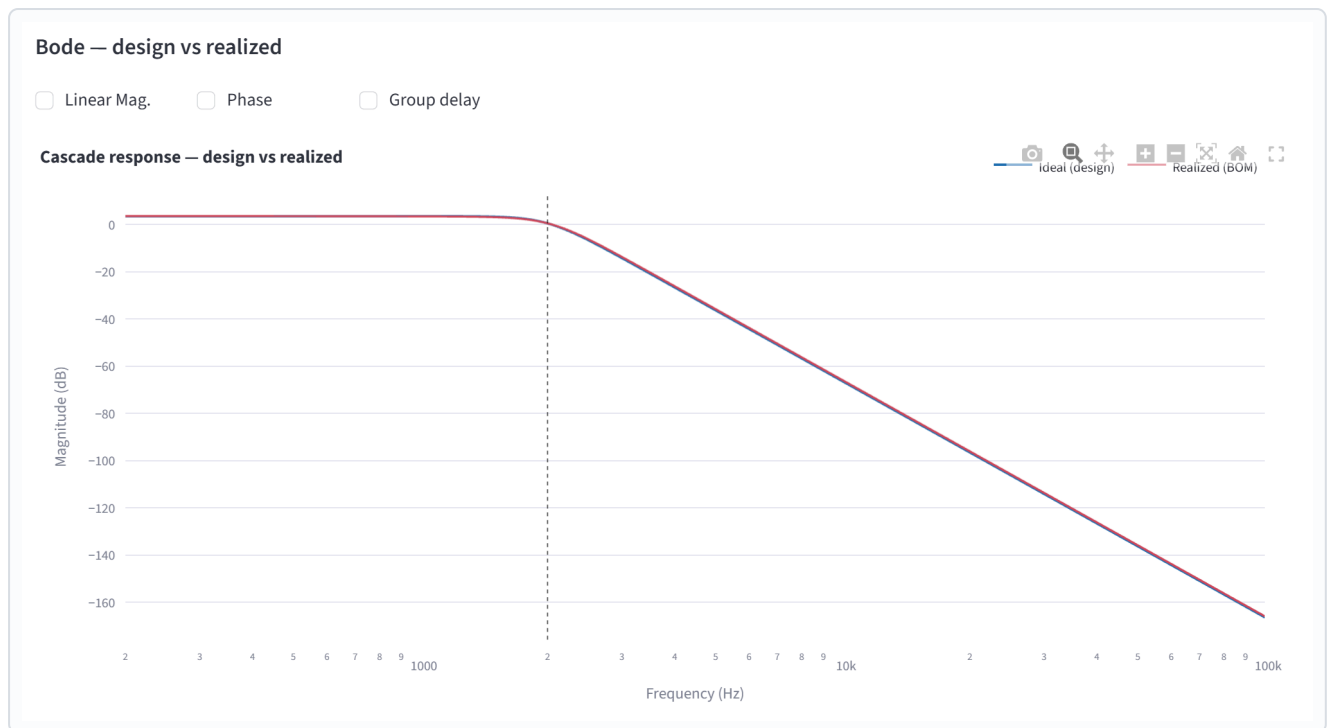


Figure 12. The realized cascade (red) on top of the design (blue).

Then tolerances. **Monte-Carlo tolerances** rebuilds the cascade 2000 times with every resistor and capacitor varied at random. For this run choose a plain spread: resistor **tol % 2**, **Capacitor tol (%) 5**, **Distribution** Uniform ($\pm$ tol), **Envelope** min-max — then press **Run Monte-Carlo**. The grey band covers every run, and the caption gives the DC-gain spread: 3.45 ... 3.71 dB around the nominal 3.58 dB. The seed is a control too, so the same inputs always give the same band.

Monte-Carlo tolerances

Per-component tolerance spread of the cascade. Resistors use value-banded tolerances (rows below); op-amp parameters are held fixed.

Resistor tolerance bands — $R_{min} \leq R < R_{max}$, in k Ω . Type a max to open the next band; blank max = ∞ .

R min (k Ω)

R max (k Ω)

tol %

Capacitor tol (%)

0

Max

2.00

Runs

2000

Distribution

☐ Gaussian (tol = 3σ)

☒ Uniform ($\pm$ tol)

Seed

0

Envelope

min-max

⚠ 2000 runs may take a few seconds.

Run Monte-Carlo

DC-gain spread over 2000 runs: 3.45 ... 3.71 dB (nominal 3.58 dB).

Bode — design vs realized

☐ Linear Mag.

☐ Phase

☐ Group delay

Cascade response — design vs realized

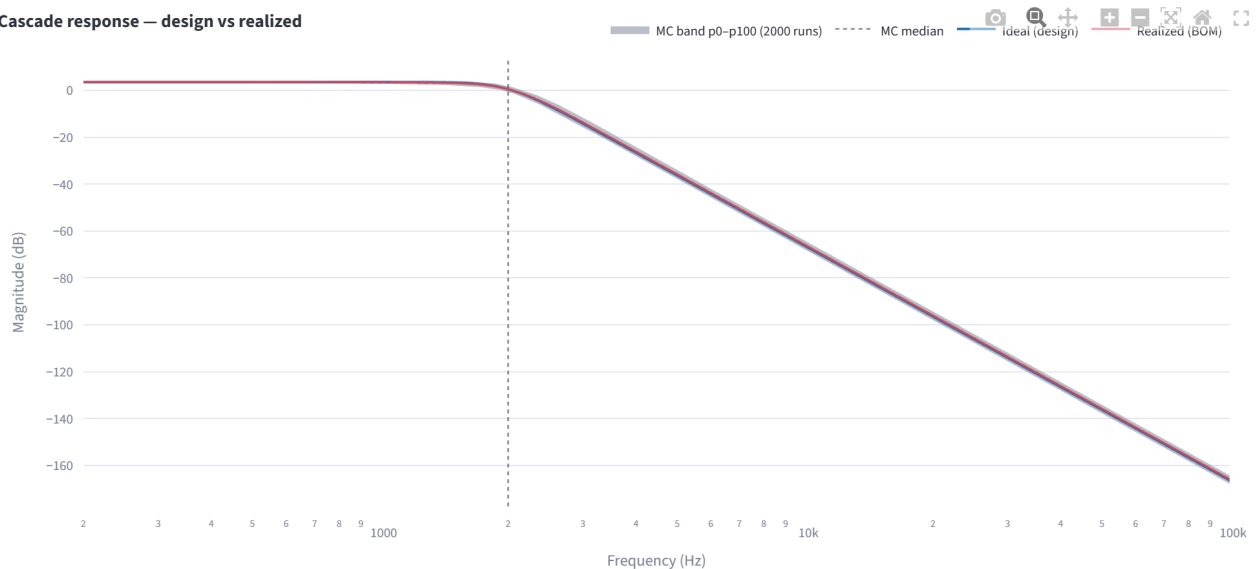


Figure 13. 2000 runs with 2 % resistors and 5 % capacitors: DC gain between 3.45 and 3.71 dB.

On a decibel axis the band is hard to see. Tick **Linear Mag.** above the plot: the spread opens up where it matters, around the corner, where tolerances move the response from about 1.25 to 1.6 V/V.

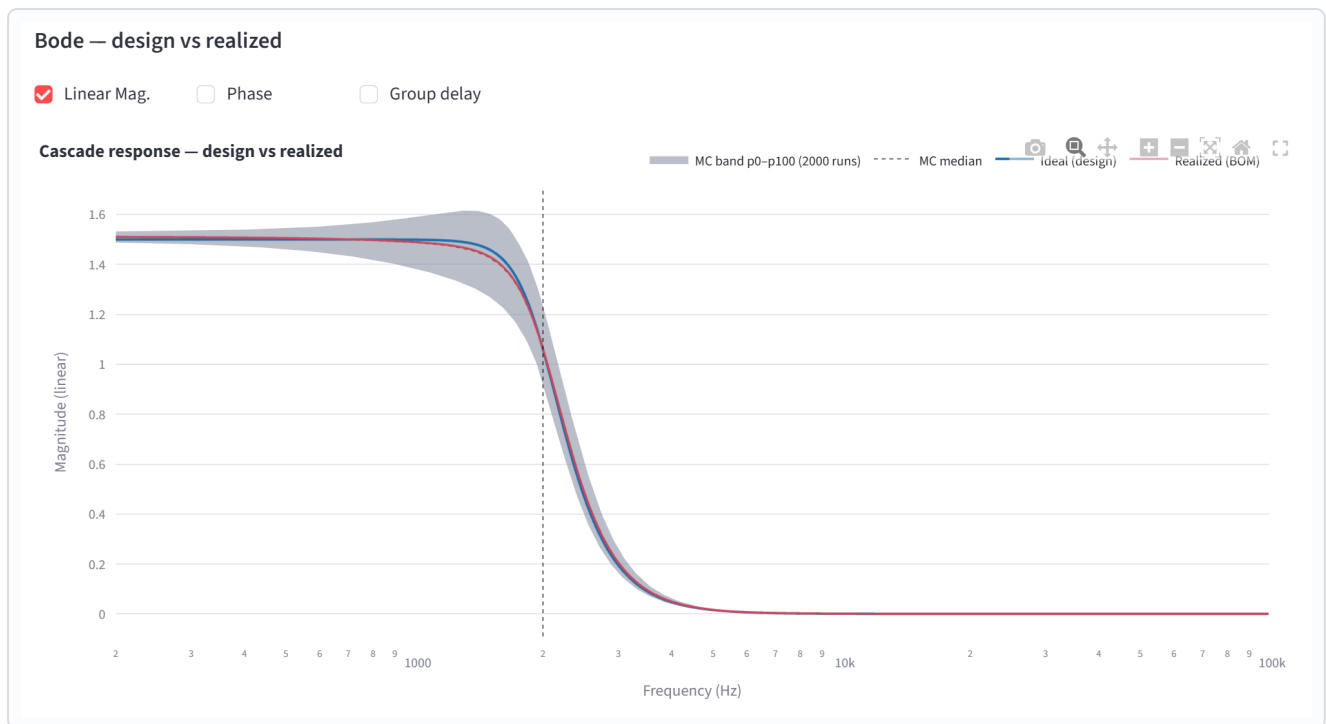


Figure 14. The same band on a linear axis: the tolerances matter most just below the corner.

7. Take the design with you

LTspice. Just above the report, the **LTspice export** block lists each section with the SPICE model its op-amp will use — the app's own generic model for an Ideal op-amp — and **↓ Download LTspice files (.zip)** saves the whole cascade as LTspice schematics and netlists, nominal and Monte-Carlo, with a README of the values LTspice should show. With Ideal op-amps the exported cascade matches the app's curve exactly. The User Manual, \$9, covers real op-amp models.

LTspice export

LTspice schematics (.asc) and netlists (.cir) of the whole cascade — sections in series, so LTspice simulates the real inter-stage loading. Each op-amp uses its part's SPICE model from the local op-amp model library, or FS generic (the tool's own $A_{ol} / GBWP / R_o$). AC nominal and AC Monte-Carlo (tolerances from the settings above), plus a README with the expected probe values.

Supply Vs (V) MC runs

Section	Cell	Op-amp (tool)	SPICE model	Drawing	DC path
1	3LP-gained	Ideal (no op-amp limits)	FS generic · A_{ol} 1 G · GBWP 10 THz · R_o 0 Ω	template SK_LP	ok
2	2LP-unity	Ideal (no op-amp limits)	FS generic · A_{ol} 1 G · GBWP 10 THz · R_o 0 Ω	template SK_LP	ok

Inter-stage loading: the exported cascade differs from the tool's realized curve (a product of unloaded sections) by at most 0.000 dB within 60 dB of the peak. The README lists the expected LTspice probe values.

2000 Monte-Carlo runs: fast to simulate, slow for LTspice to plot.

> Op-amp model library

↓ Download LTspice files (.zip)

Figure 15. The LTspice export: one row per section, and the zip.

The report. At the foot of the same tab, **Generate Report** builds an A4 PDF: the specification, the transfer function, the pole–zero map with the section pairing, a page per section with its schematic and BOM, and the design-versus-realized Bode plot with the Monte-Carlo band. Everything is ticked by default. Press  **Generate Report**, then **↓ Download PDF**.

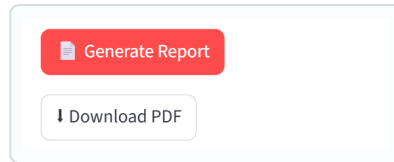


Figure 16. The report is ready to download.

Change anything afterwards and a caption says the PDF is out of date — press **Generate Report** again. Each schematic can also be saved on its own with **↓ SVG**, in the Topology tab and in **Section schematics** on this one.

Where to go next

The User Manual picks up where this guide stops:

- **§3** — every control, tab by tab, with its default — including the delay-optimized responses (Bessel, Equiripple Delay) and entering your own transfer function (Custom $H(s)$).
- **§4** — choosing between VCVS, MFB and AM for a section.
- **§5** — real op-amps, and what they change.
- **§6** — every message the app shows, and what to do about it. Start there when a solve comes back empty.
- **§8** — what each page of the report contains.
- **§9** — the LTspice export, and importing a vendor's op-amp model.